TRAINASSIST

A Smart Mobile Application for Train Passengers

Project Report

Submitted for the Partial Fulfillment of the Requirements

Bachelor of Engineering / Bachelor of Technology

Technology Stack: Flutter · ASP.NET Core · PostgreSQL

Platform: Android (Physical Device — Samsung Galaxy S23)

Date: March 2026

TABLE OF CONTENTS

Sr. No.	Chapter / Section Title
1	Abstract
1.1	Introduction
1.2	Brief Overview of Technology
1.3	Software Requirements
2	Existing System
3	Proposed System
4	Use Case Diagram
5	Domain Specification
6	Source Code & Screenshots
7	Summary
8	Conclusion
9	Future Enhancement
10	Glossary
11	Reference

1 ABSTRACT

TrainAssist is a cross-platform mobile application developed using the Flutter framework for the front-end and ASP.NET Core Minimal API with a PostgreSQL relational database for the back-end. The application addresses a real and daily challenge faced by millions of train passengers: the lack of real-time, accessible information about coach crowd levels, emergency reporting, lost-and-found coordination, and on-journey situational awareness.

The system allows passengers to search for trains by source and destination, view the crowd occupancy status (Low / Medium / High) of individual coaches as reported by fellow passengers, and submit their own crowd updates. In emergency situations, users can trigger a full SOS flow that captures GPS location, records a voice note, and dispatches alerts via SMS, WhatsApp, and Email (SMTP) to pre-configured emergency contacts. Additional features include a Bluetooth-based station proximity alert, a lost-and-found reporting module, a medical emergency profile accessible from the lock screen, and a rule-based on-device chatbot capable of answering common passenger queries such as live train tracking, PNR status, ticket cancellation, and nearest station directions.

The application was built and deployed as a debug APK on a physical Android 14 (API 34) device — Samsung Galaxy S23 — over a Wi-Fi ADB connection. The entire stack, excluding cloud deployment, is free of cost to run locally.

1.1 INTRODUCTION

Rail transport continues to be one of the primary modes of long-distance and inter-city travel across much of the world, particularly in countries like India where the railway network spans thousands of kilometres and serves hundreds of millions of passengers annually. Despite this scale, the average passenger still lacks a simple, unified digital tool to get actionable real-time information about their journey — from coach crowd conditions to emergency assistance.

TrainAssist was conceived to bridge this gap. The application puts several safety and convenience features directly in the pocket of every train passenger. By allowing passengers themselves to report crowd conditions, the system creates a community-driven, real-time occupancy map. The SOS module provides a multi-channel emergency alert system that operates even in partially connected environments, leveraging SMS and WhatsApp which work over cellular data. The chatbot immediately answers common questions without requiring an internet connection.

The project follows an N-tier architecture with a clear separation between:

- **Presentation Layer** — Flutter (Dart) mobile app targeting Android.
- **Business / API Layer** — ASP.NET Core Minimal API (.NET 10) exposing REST endpoints.
- **Data Layer** — Entity Framework Core 10 ORM connected to a PostgreSQL 15 database.
- **On-device Intelligence** — Rule-based chatbot (SimpleChatBot) running entirely offline.

Development adopted an iterative, prompt-driven workflow assisted by GitHub Copilot, accelerating feature delivery across multiple sprint-like cycles. Each prompt document (e.g., PROMPT_2_3_COMPLETE.md, PROMPT_7_8_COMPLETE.md) captures the complete implementation of a set of features.

1.2 BRIEF OVERVIEW OF TECHNOLOGY

Flutter (Dart)

Flutter is Google's open-source UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase in the Dart programming language. The framework's widget-based rendering engine provides pixel-perfect UI on Android and iOS without requiring platform bridges for most components. TrainAssist uses Flutter 3.x / Dart SDK ^3.8.1 and targets Android (primary) and Chrome web (secondary).

ASP.NET Core Minimal API (.NET 10)

Microsoft's .NET 10 platform provides a lightweight, high-performance HTTP API framework. Minimal APIs use a functional, top-level statement style to define endpoints directly in `Program.cs` without controllers, reducing boilerplate. Entity Framework Core 10 with Npgsql driver handles all database interactions using code-first migrations.

PostgreSQL 15

PostgreSQL is a powerful, open-source object-relational database system known for ACID compliance, rich data types, and mature tooling. The TrainAssist schema includes five tables: `Trains`, `Coaches`, `CrowdReports`, `SOSReports`, and `Users`. A DataSeeder class pre-populates the database with sample train and coach data on first run.

Provider (State Management)

The Flutter app uses the Provider package (ChangeNotifier pattern) for state management across eight providers: UserProvider, TrainProvider, CoachProvider, SOSProvider, BluetoothProvider, StationAlertProvider, LostFoundProvider, and MedicalProfileProvider.

Key Flutter Packages

Package	Version	Purpose
http	^1.2.2	REST API calls and multipart file upload
provider	^6.1.0	State management (ChangeNotifier)
shared_preferences	^2.2.0	Local persistence (user name, SMTP config)
intl	^0.18.0	Date & time formatting
flutter_blue_plus	^1.35.3	BLE + Classic Bluetooth scanning
geolocator	^13.0.2	GPS coordinates
geocoding	^2.0.5	Reverse geocoding (coords → address)
url_launcher	^6.3.1	Open URLs, Maps, SMS, WhatsApp
record	^5.2.0	Voice note audio recording
audioplayers	^6.1.0	Voice note playback preview
share_plus	^10.1.4	Native share sheet (WhatsApp, SMS, Email)
flutter_local_notifications	^17.2.4	Persistent medical lock-screen card
permission_handler	^11.3.1	Runtime permission management
path_provider	^2.1.4	Temp directory for voice note file path

Developer Tooling

- Visual Studio Code with Flutter and C# extensions
- GitHub Copilot (AI-assisted development)
- ADB over Wi-Fi (wireless device debugging and installation)
- Swagger UI (auto-generated API documentation at `/swagger`)
- Docker (optional — PostgreSQL container alternative)

1.3 SOFTWARE REQUIREMENTS

Development Environment

Component	Minimum Version	Notes
Flutter SDK	3.8.1	Dart SDK ^3.8.1 included
.NET SDK	10.0	Required for backend API
PostgreSQL	15	Local or Docker instance
Android Studio / SDK	API 21+	For build tools (no emulator needed)
Visual Studio Code	Latest	Recommended IDE
Node.js	18+	Optional — for tooling scripts
macOS / Linux / Windows	Any modern	All platforms supported for development

Target Runtime

Component	Specification
Mobile OS	Android 5.0 (API 21) or later; tested on Android 14 (API 34)
Backend hosting	Any machine running .NET 10 runtime; locally on port 5000
Database	PostgreSQL 15 accessible via connection string in appsettings.json

Hardware Requirements

- Android physical device (Samsung Galaxy S23 used in testing) or Android emulator
- Bluetooth 4.0+ for station proximity alert feature
- GPS sensor for location services
- Microphone for SOS voice note recording
- Internet connection (Wi-Fi or cellular) for API calls; chatbot works offline

2 EXISTING SYSTEM

Several applications and platforms currently offer partial solutions for train passengers. An analysis of the existing landscape reveals significant gaps that TrainAssist aims to address:

National Train Enquiry System (NTES – ntes.in)

NTES provides official live running status for all Indian Railways trains. However, it is a web portal without a companion mobile application offering integrated safety features. It does not provide crowd reporting, SOS alerts, or offline capabilities.

RailYatri / ixigo

These third-party platforms offer train status, PNR checking, and ticket booking. Neither provides a community-driven real-time coach occupancy system or a one-tap SOS emergency alert that integrates GPS, voice recording, SMS, and WhatsApp simultaneously.

IRCTC Official App

IRCTC is the official ticketing platform. Its scope is limited to booking and cancellation; it does not address on-journey safety, crowd conditions, or emergency response.

General Limitations of Existing Systems

- No real-time community-driven coach crowd reporting.
- No integrated multi-channel SOS (SMS + WhatsApp + Email + Voice Note).
- No Bluetooth-based station proximity and wake-up alert.
- No on-device offline chatbot answering train-related queries.
- No lost-and-found coordination within the journey context.
- No medical emergency profile stored on the device for first-responders to access.
- No single app combining all the above in a unified, privacy-first, open-source stack.

Disadvantages of the Existing System

1. Fragmented experience — passengers must switch between multiple apps.
2. Most features require internet connectivity at all times.
3. No personalised emergency configuration (contacts, SMTP, WhatsApp).
4. SOS communication relies on phone calls only; no automated multi-channel dispatch.

5. Privacy concerns with large commercial platforms handling location data.

3 PROPOSED SYSTEM

TrainAssist proposes a unified, open-source mobile application that consolidates all passenger safety and convenience features under a single interface, with a lightweight REST API backend and an on-device chatbot that works offline.

Key Features of the Proposed System

3.1 Train Search and Coach Crowd Status

- Search trains by source and/or destination station name (partial match supported).
- View all coaches for a train with the latest crowd status: Low 🟢, Medium 🟡, or High 🔴.
- Submit a new crowd report for any coach — persisted in PostgreSQL via the REST API.
- Crowd data is timestamped; the most recent report is always displayed.

3.2 SOS Emergency Alert System

- One-tap SOS trigger, or triple-tap screen gesture, or Volume-UP ×3 (Guard Mode).
- Captures GPS coordinates with reverse geocoding to a human-readable address.
- Records a voice note (M4A/WAV/Opus) and uploads it to the backend's `/api/uploads` endpoint.
- Sends the SOS record (name, train, coach, GPS, message) to the backend `/api/sos`.
- Dispatches SMS to all saved emergency contacts using the native SMS composer.
- Opens WhatsApp with a prefilled SOS message for each contact (optional auto-send via Accessibility service).
- Sends an email with the voice note attached via SMTP (configurable host, port, credentials).

3.3 Bluetooth Station Proximity Alert

- User configures a target station name and scans for nearby Bluetooth beacons.
- When a beacon matching the station name is detected, a high-priority full-screen notification is fired.
- Prevents passengers from sleeping through their stop on long-distance journeys.

3.4 Lost and Found

- Passengers can report lost or found items specifying description, location, and contact info.
- The app automatically matches lost reports with found reports and notifies both parties.

- Entirely local/on-device storage (SharedPreferences); no backend dependency.

3.5 Medical Emergency Profile

- Users enter their blood group, allergies, medical conditions, and emergency contact.
- A persistent, high-priority lock-screen notification (via `flutter_local_notifications`) keeps the medical card accessible at all times — critical when a passenger is unconscious.

3.6 On-Device Chatbot

- Rule-based bot (`SimpleChatBot`) runs entirely offline — no AI API costs.
- Handles: live train tracking links, ETA, PNR status, ticket cancellation/refund, nearest station, SOS flow, helpline numbers (139 / 182), crowd reporting guide, WhatsApp setup, and SMTP configuration.
- Returns tappable links opening the appropriate page in the browser or Google Maps.

3.7 REST API and Swagger

- ASP.NET Core Minimal API exposes fully documented endpoints at `/swagger`.
- Endpoints: GET /api/trains, GET /api/trains/{id}/coaches, POST /api/coaches/{id}/crowd, POST /api/sos, GET /api/sos, POST /api/uploads, POST /api/sendmail, GET /api/trains/{num}/links, GET /api/health.

Advantages of the Proposed System

1. Single unified app replacing five or more separate tools.
2. Community-driven real-time crowd data with no external API dependency.
3. Multi-channel SOS that works independently of phone call availability.
4. Offline chatbot answers common questions without internet.
5. Open-source, self-hosted — no subscription or usage fees.
6. Medical card always visible on lock screen — aids first-responders.
7. Bluetooth station alert eliminates missed-stop anxiety on overnight trains.

4 USE CASE DIAGRAM

The following textual representation describes the main actors and use cases of the TrainAssist system (a formal UML diagram would be rendered in a drawing tool such as draw.io or PlantUML for the final submission).

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ TrainAssist System │ │ │ │ ┌──────────┐ │ │ │ │─── Search Train
──────────────────────────────────────────► │ │ │ │─── View Coach Crowd
Status ──────────────────────────────► │ │ │ │─── Report Coach Crowd
──────────────────────────────────► │ │ │ │─── Trigger SOS
──────────────────────────────────────► │ │ │ Passenger│ └── Capture
GPS ──────────────────────────────────► │ │ │ (User) │ └── Record
Voice Note ──────────────────────────► │ │ │ │ └── Send SMS /
WhatsApp / Email ──────────────────► │ │ │ │─── Set Station Alert
(Bluetooth) ──────────────────────► │ │ │ │─── Report Lost / Found
Item ──────────────────────────► │ │ │ │─── View / Edit Medical
Profile ──────────────────────► │ │ │ │─── Ask Chatbot (offline)
──────────────────────────► │ │ └──────────┘ │ │ │ │
┌──────────────┐ │ │ │ Backend │◄── Submit crowd report ───
<<include>> ── Report Coach │ │ │ REST API │◄── Submit SOS report
─── <<include>> ── Trigger SOS │ │ │ (ASP.NET) │◄── Upload voice
note ── <<include>> ── Record VN │ │ │ │◄── Send email ──────────
<<include>> ── SOS Email │ │ └──────┬──────┘ │ │ │ │ │
┌──────▼──────┐ │ │ │ PostgreSQL │ │ │ Database │ │ │
└──────────────┘ │ │ │ │ ┌──────────┐ │ │ │ Emergency │◄──
Receives SMS / WhatsApp / Email from SOS │ │ │ Contacts │ │ │
└──────────────┘ │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Use Case Descriptions

Use Case	Actor	Description
Search Train	Passenger	Search trains by source/destination; results fetched from API
View Coach Crowd	Passenger	View latest crowd status (Low/Medium/High) per coach
Report Crowd	Passenger	Submit a new crowd report saved via POST /api/coaches/{id}/crowd
Trigger SOS	Passenger	Captures GPS, records voice, dispatches multi-channel alert
Station Alert	Passenger	Bluetooth scan triggers notification when target station is near
Lost & Found	Passenger	Submit lost/found reports; auto-match by description keywords
Medical Profile	Passenger	Edit and persist medical info shown on lock screen
Ask Chatbot	Passenger	Offline Q&A for train tracking, PNR, SOS guidance, etc.

5 DOMAIN SPECIFICATION

Application Domain

TrainAssist operates within the **railway passenger assistance** domain, intersecting public transportation, personal safety, and community informatics. The primary users are rail passengers who need real-time situational awareness and emergency support during their journey.

Data Models

Train

Field	Type	Constraint	Description
Id	int	PK, auto-increment	Unique identifier
TrainName	string(100)	Required	Name of the train
Source	string(100)	Required	Origin station
Destination	string(100)	Required	Destination station
Timing	string(10)	Required	Departure time (HH:mm)
Platform	string(10)	Optional	Platform number

Coach

Field	Type	Constraint	Description
Id	int	PK	Unique identifier
TrainId	int	FK → Train	Parent train reference
CoachName	string	Required	e.g., "S1", "B2", "General"

CrowdReport

Field	Type	Constraint	Description
Id	int	PK	Unique identifier
CoachId	int	FK → Coach	Coach being reported
ReporterName	string(100)	Required	Name of the submitting passenger
Status	string(20)	Low/Medium/High	Crowd level
Timestamp	DateTime	UTC	Time of report submission

SOSReport

Field	Type	Constraint	Description
Id	int	PK	Unique identifier
ReporterName	string(100)	Required	Name of the passenger in distress
TrainId	int?	FK (optional)	Train at time of SOS
CoachId	int?	FK (optional)	Coach at time of SOS
Latitude	double?	GPS	Latitude coordinate
Longitude	double?	GPS	Longitude coordinate
Message	string(500)	Optional	Custom SOS message
Timestamp	DateTime	UTC	Time of SOS dispatch

API Endpoint Specification

Method	Endpoint	Description
GET	/api/health	Health check — returns status + timestamp
GET	/api/trains?source=&destination=	Search trains by station
GET	/api/trains/{trainId}/coaches	Get coaches + latest crowd status
GET	/api/trains/{num}/links?date=	Get live tracking links for a train
POST	/api/coaches/{coachId}/crowd	Submit a crowd report
POST	/api/sos	Submit an SOS emergency report
GET	/api/sos	Retrieve recent SOS reports (last 50)
POST	/api/uploads	Upload voice note audio file (multipart)
POST	/api/sendmail	Send SOS email with voice note attachment

State Management Architecture

The Flutter app uses the **Provider** pattern (ChangeNotifier) with eight dedicated providers registered at the root of the widget tree in `main.dart`. Each provider owns its slice of application state and exposes methods that trigger HTTP calls to the backend API.

Provider	Responsibility
UserProvider	Persists and exposes the logged-in user name via SharedPreferences
TrainProvider	Train search results fetched from GET /api/trains
CoachProvider	Coach list and crowd reports for a selected train
SOSProvider	SOS report submission, voice note recording, GPS capture, SMS/email dispatch
BluetoothProvider	BLE scan, station name matching via flutter_blue_plus
StationAlertProvider	Alert trigger logic; fires global dialog and notification on match
LostFoundProvider	Local CRUD for lost/found items; keyword-based auto-matching
MedicalProfileProvider	Medical profile storage; triggers persistent lock-screen notification

6 SOURCE CODE & SCREENSHOTS

The following key source code excerpts illustrate the core modules of TrainAssist. The complete codebase is organised under the workspace root `/TrainAssist/`.

6.1 Project Structure

```
TrainAssist/
├── TrainAssist.Api/              ← ASP.NET Core backend
│   ├── Program.cs                ← All API endpoints (Minimal API style)
│   ├── Data/
│   │   ├── AppDbContext.cs       ← EF Core DbContext
│   │   └── DataSeeder.cs        ← Initial train/coach seed data
│   ├── Models/                   ← Train, Coach, CrowdReport, SOSReport,
User
│   ├── DTOs/                     ← Request/Response data transfer objects
│   └── Migrations/              ← EF Core migrations
│
└── train_assist_app/             ← Flutter frontend
    └── lib/
        ├── main.dart             ← App entrypoint, providers, routes
        ├── config/
        │   └── api_config.dart   ← Base URL, endpoint constants
        ├── models/               ← Dart POJOs (Train, Coach, SOSReport,
etc.)
        ├── providers/            ← 8 ChangeNotifier providers
        ├── screens/              ← All UI screens
        │   ├── search_screen.dart
        │   ├── coach_list_screen.dart
        │   ├── sos_screen.dart
        │   ├── local_chat.dart   ← Chat UI
        │   ├── chatbot_screen.dart← Train finder chatbot UI
        │   ├── station_alert_screen.dart
        │   ├── lost_found_screen.dart
        │   └── medical_profile_screen.dart
        ├── services/
        │   ├── api_service.dart
        │   ├── simple_chat_bot.dart ← Rule-based offline chatbot logic
        │   ├── chatbot_service.dart ← Train links fetcher (API + fallback)
        │   ├── faq_service.dart    ← FAQ items for chatbot UI
        │   ├── bluetooth_service.dart
        │   ├── location_service.dart
        │   └── notification_service.dart
        └── widgets/              ← Reusable UI components
```

6.2 Backend — Train Links Endpoint (Program.cs excerpt)

```
app.MapGet("/api/trains/{trainNumber}/links",
    (string trainNumber, string? date, HttpRequest request, ILogger<Program>
logger) =>
{
    var baseUrl = $"{request.Scheme}://{request.Host}";
    var dt = string.IsNullOrWhiteSpace(date)
        ? DateTime.UtcNow.ToString("yyyy-MM-dd") : date;

    var links = new[]
    {
        new { title = "NTES live train running status",
              url = "https://www.ntes.in/" },
        new { title = "RailYatri live tracker",
              url = $"https://www.railyatri.in/live-train-status?train_num=
{trainNumber}" },
        new { title = "ixigo train status",
              url = $"https://www.ixigo.com/train-
tracking/{trainNumber}/train-status" },
        new { title = "Google: train status",
              url = $"https://www.google.com/search?q=Train+
{trainNumber}+status+{dt}" },
        new { title = "Rail Madad helpline / complaint",
              url = "https://railmadad.indianrailways.gov.in/" }
    };
    return Results.Ok(new { trainNumber, date = dt, links });
});
```

6.3 Flutter — SimpleChatBot (offline intent routing)

```
static BotReply reply(String message) {
  final m = message.trim().toLowerCase();

  if (m.contains('where is my train') || m.contains('live status')) {
    return const BotReply(
      "Here are the best free sites to track your train live:",
      links: [
        BotLink('NTES — Official live running status',
'https://www.ntes.in/'),
        BotLink('RailYatri live tracker',
                'https://www.railyatri.in/live-train-status'),
        BotLink('ixigo train status', 'https://www.ixigo.com/trains'),
        BotLink('Google Maps: stations near me',
                'https://www.google.com/maps/search/railway+station+near+me'),
      ]);
  }

  if (m.contains('pnr')) {
    return const BotReply(
      "Check your 10-digit PNR status here:",
      links: [
        BotLink('IRCTC PNR status', 'https://www.irctc.co.in/'),
        BotLink('NTES PNR enquiry',

'https://www.indianrail.gov.in/enquiry/PNRSTAT/PNRStatEnquiry.html'),
      ]);
  }
  // ... more intents ...
}
```

6.4 Database Schema (Entity Framework Code-First)

```
// AppDbContext.cs
public class AppDbContext : DbContext {
    public DbSet<Train>       Trains       { get; set; }
    public DbSet<Coach>       Coaches      { get; set; }
    public DbSet<CrowdReport> CrowdReports { get; set; }
    public DbSet<SOSReport>   SOSReports   { get; set; }
    public DbSet<User>        Users        { get; set; }
}
```

6.5 SOS Flow (Provider)

```
// SOSProvider — simplified flow
async Task triggerSOS() {
  position = await LocationService.getCurrentLocation();
  voiceNoteUrl = await _recordAndUpload();
  await _submitSOSReport(position, voiceNoteUrl);
  await _sendSMSToContacts(position);
  await _openWhatsAppForContacts(position);
  await _sendEmailWithVoiceNote(position, voiceNoteUrl);
}
```

Note: Screenshots of the running application (Search screen, SOS screen, Chatbot, Station Alert, Lost & Found, Medical Profile) would be inserted here in the final printed version. Screens can be captured from the installed APK on the Samsung Galaxy S23 device.

7 SUMMARY

TrainAssist is a feature-rich, production-quality mobile application that addresses the real daily needs of train passengers through a carefully chosen technology stack. The project demonstrates:

- **Full-stack development** — Flutter/Dart mobile app + ASP.NET Core REST API + PostgreSQL.
- **State management** — Provider pattern with eight domain-specific ChangeNotifier classes.
- **Sensor integration** — GPS (geolocator), Bluetooth (flutter_blue_plus), microphone (record).
- **Multi-channel communication** — SMS, WhatsApp (url_launcher), Email (SMTP via backend).
- **Offline-first design** — Chatbot, medical profile, and lost/found operate without internet.
- **Security** — CORS configured for development; input sanitised before DB writes; no SQL injection via EF Core parameterised queries; audio uploads restricted to known MIME types.
- **Community data** — Crowd reports aggregate passenger feedback into real-time coach occupancy.
- **AI-assisted development** — GitHub Copilot accelerated feature development across all modules.

The application was successfully built, installed, and tested on a physical Android 14 device via wireless ADB. All major features — train search, crowd reporting, SOS dispatch (SMS, WhatsApp, email), Bluetooth station alert, lost-and-found, medical profile, and the chatbot — are functional.

8 CONCLUSION

The TrainAssist project successfully demonstrates how a small, focused mobile application can meaningfully improve the travel experience and safety of train passengers. By combining community-sourced data (crowd reports), device sensors (GPS, Bluetooth, microphone), multiple communication channels (SMS, WhatsApp, email), and an offline-capable intelligent assistant (rule-based chatbot), the application provides value that no single existing platform currently offers.

The choice of Flutter ensures a single Dart codebase that targets both Android and web without duplication of effort. The ASP.NET Core Minimal API provides a lightweight yet production-capable backend that supports Swagger documentation out-of-the-box. PostgreSQL offers a robust, free, and well-understood data store with strong ACID guarantees.

The iterative, AI-assisted development workflow — using GitHub Copilot with structured prompt documents — proved highly effective in accelerating feature delivery while maintaining code quality and architectural consistency. Each major feature set was delivered as a discrete, reviewable increment tracked in dedicated completion documents.

In conclusion, TrainAssist meets and exceeds its initial requirements. It is ready for further development toward public release, with a clear roadmap of enhancements outlined in the next section.

9 FUTURE ENHANCEMENT

1. **Real-time push notifications** — Integrate Firebase Cloud Messaging (FCM) to push crowd alerts and SOS acknowledgements to devices without requiring the app to be open.
2. **AI-powered chatbot upgrade** — Replace the rule-based `SimpleChatBot` with a small on-device ML model (e.g., TensorFlow Lite) or integrate with an LLM API to handle free-form natural language queries beyond keyword matching.
3. **Live train GPS tracking** — Subscribe to official API feeds (NTES/RailYatri) or crowd-source GPS pings from passengers to render a live train position on a map within the app.
4. **Authentication and user accounts** — Add OAuth 2.0 / JWT-based authentication so crowd reports and SOS records are tied to verified users, enabling gamification and accountability.
5. **iOS support** — Complete the iOS configuration (runner signing, permission plist entries) to publish on the Apple App Store alongside Android.
6. **Cloud deployment** — Deploy the ASP.NET Core API and PostgreSQL to a managed cloud platform (Azure / AWS / Railway.app) so the app functions for users across the country without a local server.
7. **Offline crowd data caching** — Cache the latest crowd reports locally so users can view coach occupancy even without an internet connection.
8. **Wearable integration** — Extend SOS trigger to paired smartwatches (using Wear OS APIs) for even faster emergency dispatch.
9. **Accessibility features** — Add voice-over support, large-text mode, and high-contrast themes for visually impaired passengers.
10. **Multi-language support** — Add Hindi, Tamil, Telugu, and other regional language localisation using Flutter's `intl` and ARB-based localisation system.

10 GLOSSARY

Term / Abbreviation	Definition
ADB	Android Debug Bridge — command-line tool to communicate with Android devices
APK	Android Package Kit — installable Android application file format
API	Application Programming Interface
ASP.NET Core	Cross-platform, high-performance framework for building web APIs by Microsoft
BLE	Bluetooth Low Energy — low-power wireless communication standard
CORS	Cross-Origin Resource Sharing — HTTP mechanism controlling cross-origin requests
Dart	Client-optimised programming language used by Flutter
DTO	Data Transfer Object — object carrying data between layers without business logic
EF Core	Entity Framework Core — .NET ORM for database access
FCM	Firebase Cloud Messaging — push notification service by Google
Flutter	Google's open-source UI toolkit for cross-platform app development
GPS	Global Positioning System — satellite-based location service
IRCTC	Indian Railway Catering and Tourism Corporation — official booking platform
JWT	JSON Web Token — compact, URL-safe access token format
NTES	National Train Enquiry System — official Indian Railways live status portal
ORM	Object-Relational Mapping — technique for converting between OO objects and DB records
PNR	Passenger Name Record — 10-digit booking reference number
PostgreSQL	Open-source object-relational database management system
Provider	Flutter state management package based on InheritedWidget and ChangeNotifier
REST	Representational State Transfer — architectural style for web services
SDK	Software Development Kit
SMTP	Simple Mail Transfer Protocol — email sending protocol
SOS	International distress signal; used in this app to denote emergency alert
Swagger	OpenAPI specification tooling providing interactive API documentation
UI/UX	User Interface / User Experience

UTC	Coordinated Universal Time — timezone-independent timestamp standard
WhatsApp	Meta's cross-platform messaging application, used for SOS dispatch

11 REFERENCE

1. Flutter — Official Documentation. Google LLC. *https://flutter.dev/docs*
2. Dart Programming Language — Official Documentation. Google LLC. *https://dart.dev/guides*
3. ASP.NET Core Documentation — Microsoft. *https://learn.microsoft.com/en-us/aspnet/core*
4. Entity Framework Core — Microsoft. *https://learn.microsoft.com/en-us/ef/core*
5. PostgreSQL 15 — Official Documentation. The PostgreSQL Global Development Group. *https://www.postgresql.org/docs/15/*
6. Provider Package — pub.dev. Remi Rousselet. *https://pub.dev/packages/provider*
7. flutter_blue_plus Package — pub.dev. *https://pub.dev/packages/flutter_blue_plus*
8. geolocator Package — pub.dev. Baseflow. *https://pub.dev/packages/geolocator*
9. url_launcher Package — pub.dev. Flutter Team. *https://pub.dev/packages/url_launcher*
10. record Package — pub.dev. *https://pub.dev/packages/record*
11. National Train Enquiry System (NTES). Ministry of Railways, Government of India. *https://www.ntes.in/*
12. RailYatri Live Train Tracker. RailYatri.in. *https://www.railyatri.in/*
13. ixigo Trains. Le Travenues Technology Ltd. *https://www.ixigo.com/trains*
14. Rail Madad Complaint Portal. Indian Railways. *https://railmadad.indianrailways.gov.in/*
15. IRCTC — Indian Railway Catering & Tourism Corp. *https://www.irctc.co.in/*
16. Swagger / OpenAPI Specification. SmartBear Software. *https://swagger.io/specification/*
17. GitHub Copilot — AI pair programming tool. GitHub Inc. *https://github.com/features/copilot*
18. Npgsql — .NET data provider for PostgreSQL. Npgsql Contributors. *https://www.npgsql.org/*
19. flutter_local_notifications Package — pub.dev. MaikuB. *https://pub.dev/packages/flutter_local_notifications*
20. share_plus Package — pub.dev. Flutter Community. *https://pub.dev/packages/share_plus*